Contribution title

First Author[1][0000-1111-2222-3333], Second Author[2,3][1111-2222-3333-4444], and Third Author[3][2222–3333-4444-5555]

[1] Princeton University, Princeton NJ 08544, USA
[2] Springer Heidelberg, Tiergartenstr. 17, 69121 Heidelberg, Germany
lncs@springer.com
http://www.springer.com/gp/computer-science/lncs
[3] ABC Institute, Rupert-Karls-University Heidelberg, Heidelberg, Germany
{abc,lncs}@uni-heidelberg.de

Resumen The abstract should briefly summarize the contents of the paper in 150–250 words. This is the English version of the abstract that provides a comprehensive overview of the research conducted, the methodology used, and the main findings obtained.

Keywords: first keyword· second keyword· another keyword

Título de la contribución

Resumen En este artículo se aborda el problema de la explicabilidad en modelos de aprendizaje automático mediante métodos de feature attribution.
En particular, se estudia una variante de los Shapley values conocida como Asymmetric Shapley Values (ASV), que permite incorporar conocimiento causal en la explicación de modelos de forma model-agnostic. A partir del análisis de su complejidad, se demuestra que el cálculo exacto de ASV es polinomial en modelos cuya distribución de entrada está representada por una red bayesiana del tipo Naive Bayes, en contraste con SHAP, que es #P-hard aún en este caso restringido.
Con el objetivo de extender estos resultados a clases más generales de redes bayesianas, se introduce una noción de clases de equivalencia sobre los órdenes topológicos del grafo causal subyacente, lo cual permite reducir drásticamente el número de permutaciones necesarias para computar ASV. Se presenta un algoritmo polinomial en el número de clases para identificarlas, y se implementa un esquema de cómputo exacto de ASV basado en estas clases. Además, se propone un nuevo método para computar en tiempo polinomial la predicción esperada de un árbol de decisión, sobre una distribución dada por una red bayesiana arbitraria, permitiendo así evaluar el algoritmo desarrollado para el cómputo de ASV en estos modelos.

Por último, se propone un algoritmo aproximado para calcular el ASV en familias de DAG's causales del tipo polytree. Para ello, se desarrolla un algoritmo de muestreo aleatorio de órdenes topológicos de polytrees. Estos resultados respaldan la viabilidad del enfoque propuesto en estructuras causales realistas, y se contrastan empíricamente con SHAP tanto en precisión como en eficiencia computacional.

Keywords: explicabilidad, árboles de decisión, Asymmetric Shapley Values (ASV), Shap, polytrees, ordenes topológicos

TODO

- Escribir todo xD

1. Introducción

En los últimos años, las capacidades de los modelos de inteligencia artificial han crecido de forma acelerada, al igual que la complejidad de sus arquitecturas. Este avance permitió resolver tareas que antes parecían inalcanzables, como la resolución de problemas matemáticos complejos por parte de modelos de lenguaje et. al, 2024. Al mismo tiempo, los modelos actuales han alcanzado escalas muy superiores a las de hace algunos años, tanto en cantidad de parámetros como en complejidad estructural Epoch AI, 2024. Esta falta de transparencia es especialmente problemática en contextos sensibles, como diagnósticos médicos o decisiones de alto impacto, donde no alcanza con obtener una predicción correcta: también es fundamental comprender cómo fue alcanzada. En este contexto surge el área de *Explainable Artificial Intelligence* (XAI), cuyo objetivo es desarrollar métodos que permitan explicar las decisiones de los modelos.

Dentro del área de explicabilidad, uno de los enfoques más influyentes es el de *feature attribution*, donde se asigna a cada variable de entrada un puntaje que refleja cuánto contribuyó a una predicción. En esta línea, *SHAP* es uno de los métodos más conocidos Lundberg y Lee, 2017. Está basado en los *Shapley values*, un concepto originado en teoría de juegos cooperativos Shapley et al., 1953. Intuitivamente, los Shapley values permiten repartir de manera justa el valor total obtenido por una coalición entre sus participantes, según cuánto aporta cada uno en promedio al incorporarse. Trasladado al aprendizaje automático, las features juegan el rol de los jugadores, y el valor asignado a cada una busca cuantificar cuánto contribuye a la predicción del modelo para una instancia particular.

A pesar de su popularidad, SHAP presenta limitaciones importantes. Por un lado, su cálculo exacto suele ser costoso, ya que requiere considerar una

cantidad exponencial de subconjuntos o permutaciones de features. De hecho, existen resultados que muestran que calcular SHAP resulta intratable incluso para modelos o distribuciones relativamente simples Arenas et al., 2021, 2023; Marzouk et al., 2025; Van den Broeck et al., 2022. Por otro lado, también existen cuestionamientos sobre la calidad de las explicaciones que produce. En particular, no siempre resulta claro cómo interpretar en inteligencia artificial los axiomas heredados de la teoría de juegos, y varios trabajos señalan que estas explicaciones pueden fallar en reflejar adecuadamente relaciones de dependencia, relevancia o causalidad entre las variables Fryer et al., 2021; Huang y Marques-Silva, 2023; Marques-Silva, 2023. Más en general, tampoco existe un consenso absoluto sobre qué debe considerarse una "explicación" en IA: muchas de las técnicas actuales son aproximaciones pragmáticas a un problema mucho más amplio, relacionado con comprensión, causalidad e interpretación humana Lipton, 2017; Miller, 2019.

Una propuesta para superar algunas de estas limitaciones son los *Asymmetric Shapley Values* (ASV), introducidos por Frye et al. Frye et al., 2019. Esta variante incorpora información causal al cálculo de los puntajes, restringiendo la atención a aquellas permutaciones de variables que respetan un grafo causal. De este modo, en lugar de tratar a todas las features simétricamente, se priorizan aquellas que aparecen causalmente antes que otras. Esto puede producir explicaciones más fieles al proceso generador de los datos y ayudar a detectar mejor fenómenos como sesgos o discriminación. Además, aunque ASV fue propuesto principalmente para mejorar la calidad explicativa, surge naturalmente la pregunta de si esta restricción adicional también puede traer beneficios desde el punto de vista computacional.

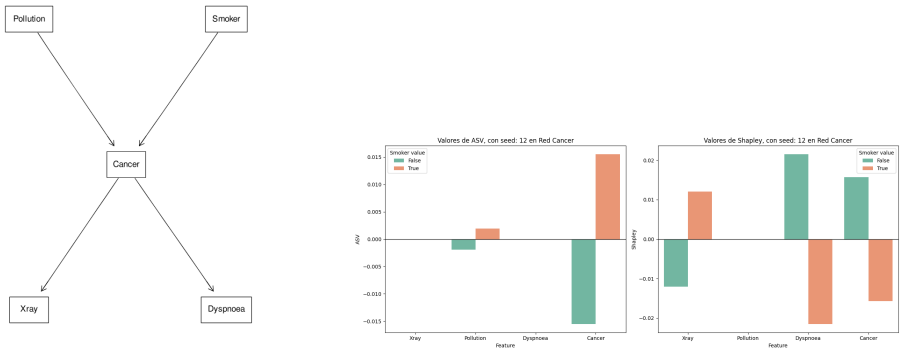
En esta tesis estudiamos el problema de calcular ASV cuando la distribución de los datos está representada por una red bayesiana. En particular, analizamos tanto el cómputo exacto como aproximado de ASV, considerando a la red bayesiana también como grafo causal subyacente. Mostramos que, en el caso de distribuciones Naive Bayes, ASV puede calcularse en tiempo polinomial en escenarios donde SHAP resulta intratable Van den Broeck et al., 2022. A partir de este resultado, nos enfocamos en árboles de decisión y en el desarrollo de algoritmos para calcular eficientemente predicciones promedio y explicaciones causales en estructuras más generales.

Echu: Esto se puede cambiar todavía, la verdad no tenemos muy clara cómo va a ser la estructura así que lo dejo cómo placeholder.

El resto del trabajo se organiza de la siguiente manera. En la Sección 2 introducimos las definiciones formales necesarias sobre SHAP, ASV, clasificadores y distribuciones probabilísticas. En la Sección 3 presentamos los principales resultados teóricos y de complejidad, junto con los algoritmos desarrollados. Luego, en la Sección 4 mostramos resultados experimentales que ilustran y validan el enfoque propuesto. Finalmente, en la Sección 5 discutimos las conclusiones y posibles líneas de trabajo futuro.

1.1. Explicaciones basadas en causalidad: un ejemplo con la red Cancer

Para ilustrar cómo la información causal puede afectar las explicaciones de un modelo, consideramos la red bayesiana *Cancer*, mostrada en la Figura 1a. En esta red, las variables `Pollution` y `Smoker` influyen sobre `Cancer`, mientras que `Xray` y `Dyspnoea` son consecuencias de esta última.



(a) Red bayesiana *Cancer*.

(b) Comparación entre SHAP y ASV.

Figura 1: Ejemplo en la red *Cancer*. SHAP puede asignar relevancia a variables que son consecuencias de otras (como `Xray` o `Dyspnoea`), mientras que ASV prioriza variables causalmente más cercanas al fenómeno.

A partir de datos sampleados de esta red entrenamos un árbol de decisión para predecir la variable `Smoker`. En este contexto podemos comparar dos formas de explicar una predicción: los valores de SHAP, que consideran todas las permutaciones posibles de features, y los Asymmetric Shapley Values (ASV), que restringen el cálculo a órdenes topológicos compatibles con la estructura causal del grafo.

La Figura 1b muestra un ejemplo de esta comparación. Se observa que SHAP asigna relevancia considerable a variables como `Xray` y `Dyspnoea`, que en la red son consecuencias de `Cancer`. Esto ocurre porque dichas variables pueden estar fuertemente correlacionadas con la predicción, aun cuando no sean causas directas del fenómeno. En cambio, ASV tiende a priorizar variables más cercanas a las causas del proceso generador de los datos, reduciendo la importancia de variables descendientes.

Este ejemplo ilustra cómo incorporar información causal puede producir explicaciones más alineadas con la estructura del problema. En redes donde se conoce (o se asume) una relación causal entre las variables, los ASVs permiten distinguir mejor entre variables que originan información relevante para la predicción y variables que simplemente reflejan efectos de otras.

2. Definiciones

En esta sección introducimos las nociones formales que utilizaremos a lo largo del trabajo. Primero definimos los *Shapley values* y su instanciación al problema de *feature attribution*. Luego presentamos los *Asymmetric Shapley Values* (ASV), que incorporan relaciones entre las variables a la atribución mediante un grafo causal. Finalmente, introducimos las redes bayesianas como modelo para la distribución de los datos y describimos las familias restringidas de redes que consideraremos en nuestros resultados.

2.1. Shapley values y feature attribution

Trabajaremos con clasificadores binarios y features binarias. Esta restricción simplifica la notación sin afectar las ideas centrales ni los algoritmos desarrollados, que pueden extenderse naturalmente a dominios finitos más generales.

Sea X un conjunto finito de features. Una *entidad* sobre X es una función $e : X \to \{0, 1\}$, donde $e(x)$ representa el valor que la entidad e toma en el feature x. Denotamos por $\texttt{ent}(X)$ al conjunto de todas las entidades posibles sobre X, y asumimos dada una distribución de probabilidad Pr sobre $\texttt{ent}(X)$. Un *clasificador binario* es una función $M : \texttt{ent}(X) \to \{0, 1\}$.

Dado un modelo M y una entidad e, un *feature attribution score* es una función $\phi : X \to \mathbb{R}$, donde $\phi(x)$ mide la relevancia del feature x respecto de la predicción $M(e)$.

Uno de los métodos de atribución más estudiados es el SHAP-SCORE Lundberg y Lee, 2017, basado en los *Shapley values* Shapley et al., 1953. En teoría de juegos cooperativos, los Shapley values describen cómo repartir el valor total obtenido por una coalición entre sus jugadores. Más precisamente, dado un conjunto finito de jugadores $\mathcal{I}$, una *función característica* es una función

$$\nu : \mathcal{P}(\mathcal{I}) \to \mathbb{R}$$

que asigna un valor a cada coalición. Los Shapley values son la única asignación que satisface simultáneamente un conjunto de axiomas clásicos: *eficiencia*, *simetría*, *jugador nulo* y *linealidad*. Una forma cerrada de definirlos es la siguiente.

Definition 1 (Shapley values). *Sea $\mathcal{I}$ un conjunto finito y sea $\nu : \mathcal{P}(\mathcal{I}) \to \mathbb{R}$ una función característica. Para cada jugador $i \in \mathcal{I}$, su valor de Shapley se define como*

$$\phi_i(\nu) = \frac{1}{|\mathcal{I}|!} \sum_{\pi \in perm(\mathcal{I})} \left[\nu(\pi_{<i} \cup \{i\}) - \nu(\pi_{<i})\right], \tag{1}$$

donde $perm(\mathcal{I})$ es el conjunto de todas las permutaciones de $\mathcal{I}$, y $\pi_{<i}$ denota el conjunto de jugadores que aparecen antes que i en la permutación π.

Intuitivamente, esta expresión promedia la contribución marginal de un jugador sobre todos los órdenes posibles de llegada al juego. De manera equivalente, puede escribirse como

$$\phi_i(\nu) = \sum_{S \subseteq \mathcal{I} \setminus \{i\}} c_{|S|} \left[\nu(S \cup \{i\}) - \nu(S) \right],$$

donde

$$c_m = \frac{m!(|\mathcal{I}| - m - 1)!}{|\mathcal{I}|!}.$$

En aprendizaje automático, el conjunto de jugadores se identifica con el conjunto de features X, en nuestro ejemplo de la red *Cancer*, X sería el conjunto `Cancer, Pollution, Dyspnoea, Xray`. Dada una entidad e y un subconjunto $S \subseteq X$, queremos evaluar el comportamiento promedio del modelo al fijar los valores de las variables de S según la entidad e y marginalizar el resto de acuerdo con la distribución de los datos.

Para ello, definimos primero el conjunto de entidades *consistentes* con e en S:

$$\mathtt{cw}(e, S) = \{e' \in \mathtt{ent}(X) : e'(x) = e(x) \text{ para todo } x \in S\}.$$

La probabilidad condicionada a este conjunto queda dada por

$$\Pr[e' \mid \mathtt{cw}(e, S)] = \begin{cases} \dfrac{\Pr[e']}{\sum\limits_{e'' \in \mathtt{cw}(e,S)} \Pr[e'']} & \text{si } e' \in \mathtt{cw}(e, S), \\ 0 & \text{en caso contrario.} \end{cases}$$

Con esta notación, la función característica asociada a M, e y Pr se define como la predicción promedio condicionada a los valores fijados en S.

Definition 2 (Función característica inducida por un modelo). *Dados un modelo M, una entidad e y una distribución* Pr *sobre* $\mathtt{ent}(X)$*, definimos*

$$\nu_{M,e,\Pr}(S) = \sum_{e' \in \mathtt{cw}(e,S)} \Pr[e' \mid \mathtt{cw}(e, S)]\, M(e'). \tag{2}$$

Esto nos permite instanciar directamente los Shapley values en el problema de *feature attribution*.

Definition 3 (SHAP). *Dado un modelo M, una entidad e, una distribución* Pr *y un feature $x_i \in X$, el valor SHAP de x_i se define como*

$$Shap_{M,e,\Pr}(x_i) = \sum_{S \subseteq X \setminus \{x_i\}} c_{|S|} \left[\nu_{M,e,\Pr}(S \cup \{x_i\}) - \nu_{M,e,\Pr}(S) \right].$$

Nótese que los axiomas que estos valores satisfacen no tienen un significado claro en el contexto de la inteligencia artificial, ya que dependen de la definición de $\nu_{M,e,\Pr}$ Fryer et al., 2021. Además, para algunas nociones simples y robustas de *feature attribution* basadas en *explicaciones abductivas* , los Shapley values no logran asignar un puntaje de 0 a features irrelevantes Huang y Marques-Silva, 2023; Marques-Silva, 2023. Aun así, constituyen uno de los marcos más influyentes para estudiar explicaciones locales basadas en atribución.

2.2. Asymmetric Shapley Values y grafos causales

La definición anterior trata a todas las permutaciones de features de manera uniforme. Sin embargo, si se dispone de un grafo causal sobre las variables, resulta natural restringir la atención a aquellas permutaciones compatibles con dicha estructura.

La generalización más simple consiste en reemplazar el promedio uniforme por una función de pesos arbitraria sobre permutaciones.

Definition 4 (Shapley values ponderados). *Sea $w : perm(\mathcal{I}) \to \mathbb{R}$ una función tal que $\sum_{\pi \in perm(\mathcal{I})} w(\pi) = 1$. Definimos*

$$\phi_i^{assym}(\nu) = \sum_{\pi \in perm(\mathcal{I})} w(\pi) \left[\nu(\pi_{<i} \cup \{i\}) - \nu(\pi_{<i}) \right]. \tag{3}$$

La familia de *Asymmetric Shapley Values* propuesta en Frye et al., 2019 surge al elegir la función de pesos a partir de un DAG que codifica relaciones causales entre las features.

Definition 5 (Asymmetric Shapley Values). *Sea $G = (X, E)$ un DAG cuyos nodos son los features. Denotemos por $topos(G)$ al conjunto de órdenes topológicos de G. Definimos la función de pesos*

$$w(\pi) = \begin{cases} \frac{1}{|topos(G)|} & \text{si } \pi \in topos(G), \\ 0 & \text{en otro caso.} \end{cases}$$

Entonces, dado un modelo M, una entidad e y una distribución $\Pr$, el valor ASV de un feature $x_i \in X$ se define como

$$Shap_{M,e,\Pr}^{assym}(x_i) = \frac{1}{|topos(G)|} \sum_{\pi \in topos(G)} \left[\nu_{M,e,\Pr}(\pi_{<i} \cup \{x_i\}) - \nu_{M,e,\Pr}(\pi_{<i}) \right].$$

La intuición detrás de esta definición es que sólo se consideran órdenes que respetan la causalidad. De este modo, si una variable x es causa de una variable y, entonces x siempre aparece antes que y en las permutaciones admitidas. Esto tiende a favorecer a las variables causalmente anteriores y a reducir la importancia de variables que son meramente consecuencias de otras.

2.3. Redes bayesianas como modelo de distribución

Para evaluar los ASV no alcanza con fijar un modelo predictivo: también es necesario especificar una distribución sobre el espacio de entidades. En este trabajo utilizamos redes bayesianas para representar dicha distribución.

Definition 6 (Red bayesiana). *Una Red Bayesiana N sobre un conjunto de variables X es una tupla $N = (X, E, \Pr)$, donde (X, E) es un DAG y, para cada variable $x \in X$, se especifica una distribución condicional $\Pr(x \mid Parents(x))$. Estas distribuciones se representan mediante tablas de probabilidad condicional (*Conditional Probability Tables*, CPTs).*

La propiedad central de una red bayesiana es que factoriza la distribución conjunta usando únicamente dependencias locales. Si $X = \{X_1, \ldots, X_n\}$, entonces

$$\Pr(X_1, \ldots, X_n) = \prod_{i=1}^{n} \Pr(X_i \mid \mathit{Parents}(X_i)).$$

Esta factorización permite realizar inferencia probabilística de forma mucho más eficiente que manipulando directamente la distribución conjunta.

Conviene distinguir con cuidado entre *red causal* y *red bayesiana*. Una red bayesiana modela dependencias condicionales y define una distribución conjunta; un grafo causal, en cambio, pretende reflejar relaciones de causa y efecto entre variables. En general, ambas nociones no coinciden necesariamente. Sin embargo, en este trabajo asumiremos que el DAG subyacente a la red bayesiana puede utilizarse también como grafo causal para caclular los ASV. Esta decisión responde a dos motivos: por un lado, es consistente con el tipo de datos y experimentos que analizamos; por otro, nos permite estudiar simultáneamente el costo de la inferencia probabilística y el efecto de imponer restricciones causales en el cálculo de explicaciones.

Más precisamente, dado un modelo M, una entidad e y una red bayesiana N sobre las features, utilizaremos la distribución inducida por N para calcular la función característica $\nu_{M,e,\Pr}$, y el DAG de N como estructura causal para definir $\mathit{Shap}^{\mathit{assym}}_{M,e,\Pr}$.

2.4. Familias restringidas de redes bayesianas

La complejidad de la inferencia en redes bayesianas depende fuertemente de la estructura del grafo subyacente. En redes generales, calcular probabilidades marginales o condicionadas es un problema #P-HARD, mientras que para ciertas familias restringidas la inferencia es tratable. Por esta razón nos centramos en grafos para los cuáles la inferencia puede realizarse en tiempo polinomial.

La primera familia que consideramos es la de las redes *Naive Bayes*. En este caso, existe una variable raíz X_1 tal que todas las demás variables son sus hijas, y no hay otras aristas. Es decir, el conjunto de aristas es $E = \{(X_1, X_j) : 2 \leq j \leq n\}$.. La distribución conjunta factoriza entonces como

$$\Pr[e] = \Pr[X_1 = e(X_1)] \prod_{j=2}^{n} \Pr[X_j = e(X_j) \mid X_1 = e(X_1)].$$

Echu: Hace falta mostrar la fórmula de la probabilidad o mejor mostrar la imagen de que estructure tiene y listo? Igual la imagen ocupa espacio y no suma tanto, tal vez no hace falta ninguna de las dos.

Esta familia es especialmente relevante porque, aunque su estructura es muy simple, ya permite exhibir una diferencia fundamental entre SHAP y ASV: se sabe que calcular SHAP sobre distribuciones Naive Bayes es intratable incluso para modelos muy simples Van den Broeck et al., 2022, mientras que uno de nuestros resultados muestra que ASV puede ser tratable en este escenario.

La segunda familia central en este trabajo es la de los *polytrees*, es decir, DAGs cuyo grafo subyacente no dirigido es un árbol. Esta clase resulta importante porque admite inferencia probabilística en tiempo polinomial Pearl, 1986. Como el cálculo de la función característica involucra predicciones promedio condicionadas a evidencia, la posibilidad de realizar inferencia eficiente en la distribución de entrada es un ingrediente fundamental para obtener algoritmos tratables para ASV.

Echu: ¿Se dice tratable o tractable?

En síntesis, a lo largo de esta tesis trabajaremos principalmente con dos tipos de restricciones estructurales sobre la distribución de los datos: redes Naive Bayes, que sirven como caso base para nuestros resultados de tractabilidad, y polytrees, que constituyen la clase más general sobre la que desarrollamos algoritmos eficientes para inferencia y cómputo de explicaciones.

Echu: ¿Este parrafo juega o este de más?

3. Resultados teóricos

3.1. Tratabilidad de ASV

Como vimos en la sección anterior, la complejidad de la distribución de entrada no es el único factor que determina la dificultad de calcular explicaciones basadas en Shapley values. De hecho, aun para distribuciones estructuralmente simples como las redes Naive Bayes, el cálculo exacto de SHAP sigue siendo intratable Van den Broeck et al., 2022. En contraste, los Asymmetric Shapley Values (ASV) restringen las permutaciones consideradas a aquellas compatibles con el grafo causal. En esta sección mostramos que esta restricción permite obtener tractabilidad en casos donde SHAP no la tiene.

Consideramos el caso en que la distribución de los datos está dada por una red bayesiana Naive Bayes, y tomamos su DAG subyacente como grafo causal para calcular ASV. Recordemos que, en este caso, toda permutación topológica válida debe ubicar a la variable raíz X_1 antes que cualquier otra variable.

Esta observación permite caracterizar explícitamente el conjunto de órdenes topológicos y explotar la estructura de la distribución en el cálculo de ASV.

Theorem 1. *Sea $\mathcal{F}$ una familia de modelos. Los Asymmetric Shapley Values pueden calcularse en tiempo polinomial para distribuciones Naive Bayes si y solo si los Shapley values pueden calcularse en tiempo polinomial para $\mathcal{F}$ bajo distribuciones producto.*

Echu: La demo es larga, la ponemos completa o damos una intuición la misma? Habría que ver el tema de si podemos mandar cosas al apéndice.

Demostración. Primero, probamos la implicación de derecha a izquierda. Sea x_1 el padre de todos los demás nodos en el DAG. Vamos a mostrar cómo calcular $Shap^{assym}_{M,e,\Pr}(x_j)$ para cualquier $2 \leq j \leq n$ y $Shap^{assym}_{M,e,\Pr}(x_1)$ de forma independiente.

Observemos que el DAG tiene $(n-1)!$ órdenes topológicos, uno para cada permutación de los features $\{x_2, \ldots, x_n\}$, y $\pi(x_1) = 1$ para todas ellas. Entonces,

$$Shap^{assym}_{M,e,\Pr}(X_j) = \sum_{\pi \in topos(G)} w(\pi) \left[\nu_{M,e,\Pr}(\pi_{<j} \cup \{x_j\}) - \nu_{M,e,\Pr}(\pi_{<j})\right]$$

es equivalente a:

$$Shap^{assym}_{M,e,\Pr}(X_j) = \frac{1}{(n-1)!} \sum_{\pi \in perm(\{x_2,\ldots,x_n\})} \left[\nu_{M,e,\Pr}(\{x_1\} \cup \pi_{<j} \cup \{x_j\}) - \nu_{M,e,\Pr}(\{x_1\} \cup \pi_{<j})\right]$$

Así, una vez que x_1 está fijado, la distribución para las variables $x_2, \ldots, x_n$ es una distribución producto con $p_{x_j} = P(X_j = 1 | X_1 = e(x_1))$. Para simplificar, asumamos que $e(x_1) = 1$, y consideremos la distribución de producto $\Pr'$ definida como:

$$\Pr{}'[X_i = 1] = p_i = \begin{cases} 1 & i = 1 \\ \Pr[X_i = 1 | X_1 = e(x_1)] & \text{en otro caso} \end{cases}$$

que intuitivamente se obtiene de Pr fijando $X_1 = 1$. Siempre que x_1 esté fijo, ambas distribuciones Pr y $\Pr'$ se comportan de la misma forma:

Lemma 1. *Para cualquier $S \subseteq X \setminus \{x_1\}$, se cumple que:*

$$\nu_{M,e,\Pr}(\{x_1\} \cup S) = \nu_{M,e,\Pr'}(\{x_1\} \cup S) = \nu_{M,e,\Pr'}(S)$$

Demostración. Esto se sigue de directamente manipulando la expresión:

$$\begin{aligned}
\nu_{M,e,\Pr}(\{x_1\} \cup S) &= \sum_{e' \in \mathtt{cw}(e,\{x_1\} \cup S)} \Pr[e'|S] M(e') \\
&= \sum_{e' \in \mathtt{cw}(e,\{x_1\} \cup S)} \left(\prod_{\substack{e'(y)=1 \\ y \notin \{x_1\} \cup S}} p_y \prod_{\substack{e'(y)=0 \\ y \notin \{x_1\} \cup S}} (1 - p_y) \right) M(e') \quad (4) \\
&= \sum_{e' \in \mathtt{cw}(e,S)} \left(\prod_{\substack{e'(y)=1 \\ y \notin S}} p_y \prod_{\substack{e'(y)=0 \\ y \notin S}} (1 - p_y) \right) M(e') \\
&= \sum_{e' \in \mathtt{cw}(e,S)} \Pr{}'[e'|S] M(e') \\
&= \nu_{M,e,\Pr'}(S)
\end{aligned}$$

donde la segunda igualdad viene de reemplazar Pr por la distribución producto y la tercera igualdad se deduce al observar que para todas las entidades $e' \in \mathtt{cw}(e, S)$ tales que $e'(x_1) = 0$ se cumple que

$$\prod_{\substack{e'(y)=1 \\ y \notin S}} p_y \prod_{\substack{e'(y)=0 \\ y \notin S}} (1-p_y) = 0$$

Además, la segunda ecuación es igual a $\nu_{M,e,\mathrm{Pr}'}(\{x_1\} \cup S)$.

Usando este lema, ahora demostramos

$$Shap^{assym}_{M,e,\mathrm{Pr}}(x_j) = Shap_{M,e,\mathrm{Pr}'}(x_j) \tag{5}$$

Se cumple que

$$\begin{aligned}
Shap_{M,e,\mathrm{Pr}'}(x_j) &= \frac{1}{n!} \sum_{\pi \in perm(X)} [\nu_{M,e,\mathrm{Pr}'}(\pi_{<j} \cup \{x_j\}) - \nu_{M,e,\mathrm{Pr}'}(\pi_{<j})] \\
&= \frac{1}{n!} \sum_{\pi \in perm(X)} [\nu_{M,e,\mathrm{Pr}'}(\{x_1\} \cup \pi_{<j} \cup \{x_j\}) - \nu_{M,e,\mathrm{Pr}'}(\{x_1\} \cup \pi_{<j})] \\
&= \frac{n}{n!} \sum_{\pi \in perm(\{x_2,\ldots,x_n\})} [\nu_{M,e,\mathrm{Pr}'}(\{x_1\} \cup \pi_{<j} \cup \{x_j\}) - \nu_{M,e,\mathrm{Pr}'}(\{x_1\} \cup \pi_{<j})] \\
&= \frac{1}{n-1!} \sum_{\pi \in perm(\{x_2,\ldots,x_n\})} [\nu_{M,e,\mathrm{Pr}}(\{x_1\} \cup \pi_{<j} \cup \{x_j\}) - \nu_{M,e,\mathrm{Pr}}(\{x_1\} \cup \pi_{<j})] \\
&= Shap^{assym}_{M,e,\mathrm{Pr}}(x_j)
\end{aligned}$$

donde la segunda y cuarta igualdad se siguen del Lema 1, la última igualdad de la Ecuación 3.1 y la tercera al observar que para cada permutación de $perm(\{x_2, \ldots, x_n\})$ podemos construir n permutaciones de $perm(X)$ insertando x_1 en todos los lugares posibles, y que para cada una de estas permutaciones la expresión dentro de la suma es la misma. Así, la Ecuación 5 muestra que calcular $Shap^{assym}_{M,e,\mathrm{Pr}}(x_j)$ se reduce a calcular los valores Shapley habituales para una distribución independiente particular.

Ahora consideramos $Shap^{assym}_{M,e,\mathrm{Pr}}(x_1)$. Observemos que

$$\begin{aligned}
Shap^{assym}_{M,e,\mathrm{Pr}}(x_1) &= \frac{1}{|topos(G)|} \sum_{\pi \in topos(G)} [\nu_{M,e,\mathrm{Pr}}(\pi_{<1} \cup \{x_1\}) - \nu_{M,e,\mathrm{Pr}}(\pi_{<1})] \\
&= \frac{1}{|topos(G)|} (\nu_{M,e,\mathrm{Pr}}(\{x_1\}) - \nu_{M,e,\mathrm{Pr}}(\emptyset))
\end{aligned}$$

ya que para todas las permutaciones el conjunto $\pi_{<1} = \emptyset$. Además, sabemos que $\nu_{M,e,\mathrm{Pr}}(\{x_1\}) = \nu_{M,e,\mathrm{Pr}'}(\emptyset)$ por el Lema 1, y que

$$
\begin{aligned}
\nu_{M,e,\Pr}(\emptyset) &= \sum_{e' \in \texttt{ent}(X)} \Pr[e] M(e) \\
&= \sum_{e' \in \texttt{ent}(X) \quad e'(x_1)=1} \Pr[e'|e'(x_1)=1] \Pr[X_1=1] M(e') + \sum_{\substack{e' \in \texttt{ent}(X) \\ e'(x_1)=0}} \Pr[e'|e'(x_1)=1] \Pr[X_1=0] M(e') \\
&= \Pr[X_1=1] \nu_{M,e,\Pr'}(\emptyset) + \Pr[X_1=0] \nu_{M,e,\Pr''}(\emptyset)
\end{aligned}
$$

donde la distribución $\Pr''$ es la distribución producto obtenida al fijar $X_1 = 0$ como

$$
\Pr{}''[X_i = 1] = \begin{cases} 0 & i = 0 \\ Pr[X_i = 1 | X_1 = 0] & \text{de otro modo} \end{cases}
$$

Finalmente,

$$
Shap^{assym}_{M,e,\Pr}(x_1) = \frac{1}{|topos(G)|}((1 - \Pr[X_1 = 1]) \nu_{M,e,\Pr'}(\emptyset) - \Pr[X_1 = 0] \nu_{M,e,\Pr''}(\emptyset))
$$

y reducimos el problema de calcular $Shap^{assym}_{M,e,\Pr}(x_1)$ al de calcular la predicción promedio del modelo M para dos distribuciones independientes diferentes $\Pr'$ y $\Pr''$. Según Van den Broeck et al., 2022[Teorema 1], se cumple que estos promedios pueden ser calculados en tiempo polinomial si y sólo si los valores de Shapley pueden ser calculados en tiempo polinomial para una distribución de producto arbitraria.

La demostración de izquierda a derecha sigue al observar que una distribución producto es un caso particular de una Distribución Naive Bayes en la cual, para cualquier valor del padre X_1, las distribuciones condicionales son las mismas.

El Teorema 1 muestra que, a diferencia de SHAP, ASV puede ser tratable al explotar la estructura causal. En particular, esta tratabilidad depende de poder calcular eficientemente la predicción promedio. En la experimentación nos enfocamos en árboles de decisión, donde desarrollamos algoritmos polinomiales para esta tarea.

Echu: Tiene sentido el teaser de la experimentación? O saco esa oración?

3.2. Clases de equivalencia y relación con los órdenes topológicos

Recordemos la definición de ASV:

$$
Shap^{assym}_{M,e,\Pr}(x_i) = \frac{1}{|topos(G)|} \sum_{\pi \in topos(G)} \left[\nu_{M,e,\Pr}(\pi_{<i} \cup \{x_i\}) - \nu_{M,e,\Pr}(\pi_{<i}) \right].
$$

Para simplificar la notación, en esta sección tomamos M, e y $\Pr$ fijos, y escribimos simplemente ν en lugar de $\nu_{M,e,\Pr}$. La idea es reducir la cantidad

de órdenes topológicos sobre los cuales evaluamos ν, agrupando aquellos que devuelven el mismo resultado al evaluarse sobre ν.

La relación ideal sería identificar dos órdenes topológicos $\pi^1, \pi^2 \in topos(G)$ cuando

$$\nu(\pi^1_{<i}) = \nu(\pi^2_{<i}),$$

pero esta condición es costosa de verificar, ya que requiere evaluar explícitamente la función característica. Por este motivo trabajamos con una relación más fina, puramente combinatoria.

Definition 7. *Sea $G = (V, E)$ un DAG y sea $x_i \in V$. Definimos la relación R sobre $topos(G)$ por*

$$\pi^1 R \pi^2 \iff \{\pi^1_{<i}\} = \{\pi^2_{<i}\}.$$

Es decir, dos órdenes topológicos son equivalentes si tienen exactamente el mismo conjunto de nodos a la izquierda de x_i.

Esta relación depende únicamente de la posición relativa de los nodos respecto de x_i. Para representarla, resulta útil dividir los nodos del grafo en ancestros, descendientes y nodos no relacionados con x_i.

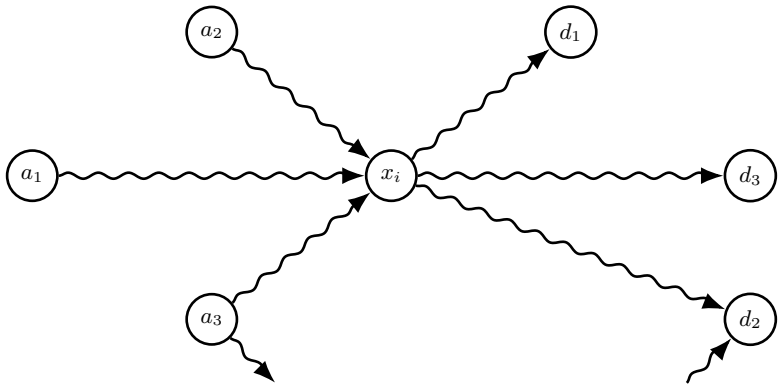


Figura 2: Al fijar un nodo x_i, dividimos el resto en tres grupos: ancestros (a), descendientes (d) y nodos no relacionados. Los ancestros siempre quedan a la izquierda de x_i, los descendientes siempre a la derecha, y solo los nodos no relacionados no tienen una posición fija respecto de x_i entre distintos órdenes topológicos.

Definition 8. *Sea $G = (V, E)$ un DAG y $x_i \in V$. Una clase de equivalencia $[\pi]_R$ queda determinada por una función*

$$f_\pi : V \setminus \{x_i\} \to \{left, right\},$$

donde $f_\pi(v)$ indica si el nodo v aparece a la izquierda o a la derecha de x_i en un orden topológico π. Denotamos por $L([\pi]_R)$ y $R([\pi]_R)$ a los conjuntos de nodos etiquetados con left y right, respectivamente.

Proposition 1. *Sea $eqCl(G, x_i)$ el conjunto de clases de equivalencia inducidas por R. Entonces*

$$Shap^{assym}_{M,e,\Pr}(x_i) = \frac{1}{|topos(G)|} \sum_{[\pi]_R \in eqCl(G,x_i)} |[\pi]_R| \left[\nu(L([\pi]_R) \cup \{x_i\}) - \nu(L([\pi]_R))\right].$$

Demostración. Basta particionar la suma original de ASV por clases de equivalencia. Dentro de una misma clase, todos los órdenes topológicos inducen el mismo conjunto de nodos antes de x_i, y por lo tanto la expresión entre corchetes es constante.

Echu: ¿Es muy vende humo ponerle demostración? Sino lo escribo cómo un parrafo y le pongo que es la intuición.

La ventaja de estas clases es que no requiere evaluar ν para decidir si dos órdenes pertenecen a la misma. La Proposición 1 muestra que, si podemos calcular eficientemente las clases de equivalencia y sus tamaños, entonces podemos computar ASV sin enumerar todos los órdenes topológicos.

3.3. Algoritmo exacto basado en clases de equivalencia

Una solución naive consiste en generar todos los órdenes topológicos de G, agruparlos según la relación R, y para cada grupo almacenar un representante y su cardinalidad. Este procedimiento es correcto, pero hereda el costo de enumerar explícitamente todos los órdenes topológicos.

Nuestro algoritmo evita esa enumeración. La idea es trabajar directamente sobre los nodos no relacionados con x_i, ya que los ancestros y descendientes tienen posición fija respecto de x_i. En un *dtree*, un polytree en el que cada nodo tiene a lo sumo un padre, estos nodos no relacionados se descomponen en una colección de subárboles disjuntos, cuyas clases de equivalencia pueden calcularse recursivamente y luego combinarse, los podemos ver en la Figura 3.

Sea UR el conjunto de raíces de los subárboles no relacionados. Nuestro algoritmo primero computa, para cada $ur \in UR$, las clases de equivalencia locales del subárbol con raíz en ur. Cada clase se representa mediante una tupla

$$(\mathsf{Rep}([\pi]_R), leftTopos, rightTopos),$$

donde $\mathsf{Rep}([\pi]_R)$ codifica qué nodos quedan a izquierda y derecha de x_i, y $leftTopos$, $rightTopos$ cuentan cuántos órdenes topológicos son compatibles con la parte izquierda y con la parte derecha, respectivamente. El tamaño total de la clase se obtiene multiplicando estas dos cantidades.

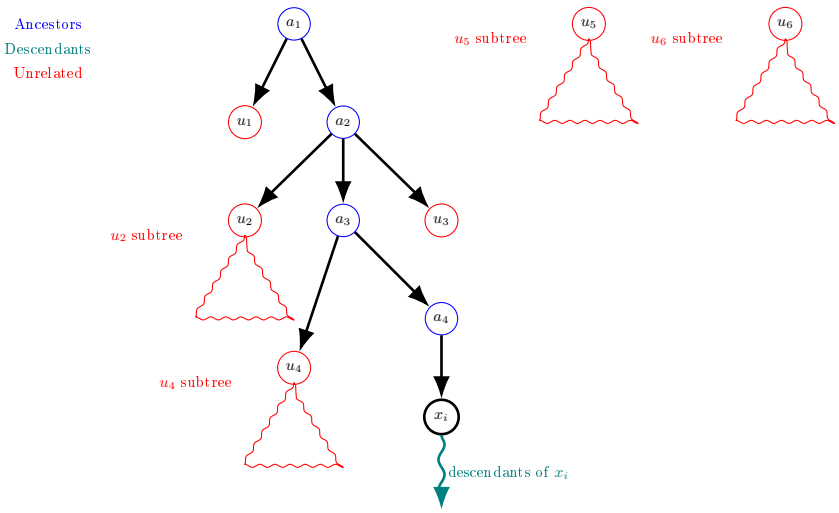


Figura 3: Ejemplo de un grafo causal G, con nodos etiquetados como ancestros, descendientes y no relacionados con respecto a x_i.

La idea recursiva es: si n es una hoja, existen exactamente dos clases: una con n a la izquierda de x_i y otra con n a la derecha. Si n no es hoja, entonces:

- si n se ubica a la derecha de x_i, todos sus descendientes quedan necesariamente a la derecha;
- si n se ubica a la izquierda de x_i, las clases de equivalencia de sus hijos pueden combinarse libremente, pues no hay aristas entre subárboles hermanos.

Esto induce la siguiente recurrencia.

Theorem 2. *Sea n un nodo de un subárbol no relacionado. Entonces la cantidad de clases de equivalencia con respecto a x_i satisface*

$$\#EC(n) = \begin{cases} 2 & \textit{si } n \textit{ es una hoja,} \\ \prod_{c \in children(n)} \#EC(c) + 1 & \textit{en otro caso.} \end{cases}$$

Echu: Acá no hace falta "demo" ni más intuición, no? Medio que la idea la explique antes

La Figura 4 ilustra una clase de equivalencia de un subárbol no relacionado. En un *dtree* G, el cálculo exacto de $eqCl(G, x_i)$ puede descomponerse en dos etapas: en primer lugar, el cómputo recursivo de las clases de equivalencia de cada subárbol no relacionado y, en segundo lugar, la combinación de dichas clases con los ancestros y descendientes de x_i para obtener sus tamaños globales.

Una vez obtenidas las clases locales de cada árbol no relacionado, las combinamos para construir las clases globales. La combinación con los descendientes

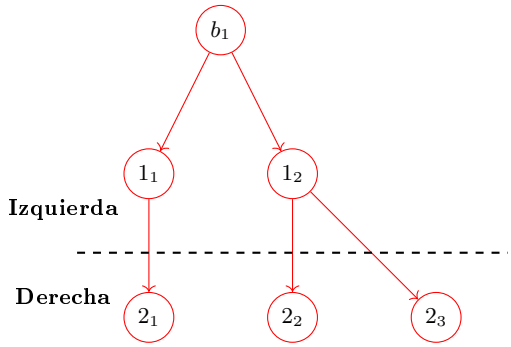


Figura 4: Ejemplo de una clase de equivalencia para un subárbol no relacionado. En este caso, $L([\pi]_R) = \{b_1, 1_1, 1_2\}$ y $R([\pi]_R) = \{2_1, 2_2, 2_3\}$.

es directa, pues no introduce restricciones adicionales entre subárboles. La parte delicada es la combinación con los ancestros: no podemos intercalarlos arbitrariamente con los nodos no relacionados que quedan a la izquierda, ya que existen dependencias de precedencia. Para resolver esto introducimos una rutina dinámica, `leftOrders`, que cuenta cuántos órdenes topológicos son compatibles con una clase dada. Esta es la base del algoritmo exacto que utilizamos para computar las clases de equivalencia.

Echu: Acá queda medio desconexo, lo divido en otra subsección o al pedo?

Para calcular los ordenamientos de los descendientes necesitamos poder calcular el número de órdenes topológicos de ese subárbol. Si t es un *dtree* con subárboles hijos $t_1, \ldots, t_r$, donde t_j tiene tamaño k_j, entonces los órdenes topológicos de los subárboles pueden intercalarse libremente, y obtenemos la siguiente recurrencia.

Definition 9. *Dado un* dtree t*, con subárboles hijos* $t_1, \ldots, t_r$ *de tamaños* $k_1, \ldots, k_r$*, la cantidad de órdenes topológicos satisface*

$$\#topos(t) = \binom{k_1 + \cdots + k_r}{k_1, \ldots, k_r} \prod_{j=1}^{r} \#topos(t_j).$$

Esta fórmula no se extiende directamente a polytrees generales, ya que allí puede haber solapamientos entre descendientes de distintas raíces. Por esta razón, en esta sección nos restringimos al caso de *dtrees*, que ya resulta suficiente para desarrollar un algoritmo exacto por clases de equivalencia.

El segundo ingrediente es la cantidad de clases de equivalencia. La recurrencia del Teorema 2 permite obtener una cota estructural sencilla.

Theorem 3. *Sea* T *un* dtree *de altura* h *y con* l *hojas. Entonces*

$$\#EC(T) \leq h^l + 1.$$

Echu: ¿Ponemos la demo? Es de media carilla

Este resultado muestra que la cantidad de clases de equivalencia depende de la forma del árbol, y no directamente del número total de órdenes topológicos, que puede ser factorial.

Pasemos ahora a la complejidad del algoritmo exacto. Sea $n = |V(G)|$ y sea $equivalenceClasses = eqCl(G, x_i)$.

Proposition 2. *La rutina UnrEC, que computa las clases de equivalencia de los subárboles no relacionados, tiene complejidad temporal*

$$O(n^2 \cdot |equivalenceClasses|).$$

Demostración (Idea de la prueba). La rutina visita cada nodo una única vez y, en cada estado, el costo dominante proviene de combinar clases de los hijos. La operación de combinación cuesta $O(n)$ y el número de combinaciones posibles está acotado por $|equivalenceClasses|$.

La fase global del algoritmo consiste en fusionar las clases locales y calcular sus tamaños incorporando ancestros y descendientes.

Proposition 3. *La complejidad temporal total del algoritmo exacto para calcular las clases de equivalencia y sus tamaños en un* dtree *es*

$$O(n^5 \cdot |equivalenceClasses|^3).$$

Demostración (Idea de la prueba). Luego de computar las clases locales con UnrEC, para cada clase se ejecuta una rutina eqClassTopos que incorpora el costo de combinar ancestros, descendientes y nodos no relacionados. El análisis detallado de esta rutina da una cota $O(n^5 \cdot |equivalenceClasses|^2)$ por clase. Multiplicando por la cantidad total de clases obtenemos la cota final.

La Proposición 3 muestra que el algoritmo exacto es polinomial en el tamaño del grafo y en el número de clases de equivalencia. Esto contrasta con la estrategia naive, cuya complejidad depende del número total de órdenes topológicos. En particular, cuando $|equivalenceClasses| \ll |topos(G)|$, el uso de clases de equivalencia produce una mejora significativa para el cómputo exacto de ASV.

Finalmente, observamos que esta técnica no requiere aproximar el valor de ASV: la reducción de costo proviene únicamente de reagrupar órdenes topológicos que inducen el mismo conjunto de nodos a la izquierda de x_i. Por ello, en los casos donde el número de clases es moderado, obtenemos una mejora exacta y no meramente heurística.

3.4. Aproximación de ASV mediante sampleo

En las secciones anteriores nos concentramos en algoritmos exactos para calcular ASV, ya sea enumerando todas las clases de equivalencia o explotando su estructura para reducir la cantidad de evaluaciones de la función característica.

Sin embargo, cuando el número de clases o de órdenes topológicos es demasiado grande, incluso estos enfoques pueden volverse costosos. Esto motiva considerar algoritmos aproximados.

La idea más natural consiste en reemplazar la suma exacta que define ASV por un promedio empírico sobre una muestra aleatoria. Existen dos variantes inmediatas de esta estrategia. La primera consiste en samplear órdenes topológicos del DAG causal G y estimar el valor de ASV a partir de sus contribuciones marginales. La segunda consiste en samplear clases de equivalencia y utilizar un representante de cada clase para aproximar el promedio exacto.

En ambos casos, el punto central es el mismo: si disponemos de un mecanismo que permita generar muestras de manera uniforme, entonces el valor de ASV puede estimarse con buena precisión a partir de una cantidad moderada de muestras. En particular, el error del estimador decrece con la cantidad de muestras tomadas, por lo que el problema se reduce a diseñar procedimientos eficientes para samplear órdenes topológicos o clases de equivalencia, y a contar con un algoritmo que permita evaluar la función característica en cada muestra.

El caso de órdenes topológicos resulta especialmente natural, ya que

$$Shap^{assym}_{M,e,\Pr}(x_i) = \frac{1}{|topos(G)|} \sum_{\pi \in topos(G)} [\nu_{M,e,\Pr}(\pi_{<i} \cup \{x_i\}) - \nu_{M,e,\Pr}(\pi_{<i})]$$

es simplemente el promedio uniforme de la variable aleatoria

$$Z_\pi = \nu_{M,e,\Pr}(\pi_{<i} \cup \{x_i\}) - \nu_{M,e,\Pr}(\pi_{<i}), \qquad \pi \sim \mathrm{Unif}(topos(G)).$$

Por lo tanto, si podemos samplear órdenes topológicos uniformemente, entonces podemos estimar ASV promediando copias i.i.d. de Z_π.

De manera análoga, la reformulación exacta basada en clases de equivalencia permite escribir

$$Shap^{assym}_{M,e,\Pr}(x_i) = \frac{1}{|topos(G)|} \sum_{[\pi]_R \in eqCl(G,x_i)} |[\pi]_R| \, [\nu_{M,e,\Pr}(L([\pi]_R) \cup \{x_i\}) - \nu_{M,e,\Pr}(L([\pi]_R))] \, .$$

Esto sugiere una segunda estrategia de aproximación: samplear clases de equivalencia con probabilidad proporcional a su tamaño y estimar el valor mediante el promedio de sus representantes. En este caso, el desafío principal consiste en disponer de un algoritmo eficiente para generar clases de equivalencia con la distribución correcta.

3.5. Resultados generales de complejidad

La utilidad de los enfoques anteriores depende de dos ingredientes algorítmicos: la capacidad de samplear de manera eficiente y la capacidad de evaluar la función característica sobre cada muestra. Esta observación se formaliza en los siguientes resultados generales.

Theorem 4. *Sea G un DAG y sea x_i un feature. Supongamos que existe un algoritmo que samplea un orden topológico uniforme de G en tiempo X, y que la cantidad*

$$\nu_{M,e,\mathrm{Pr}}(S)$$

puede calcularse en tiempo Y para cualquier subconjunto $S \subseteq X$. Entonces, para cualquier $\varepsilon, \delta \in (0,1)$, existe un algoritmo que aproxima $Shap^{assym}_{M,e,\mathrm{Pr}}(x_i)$ con error aditivo a lo sumo ε y probabilidad al menos $1-\delta$ en tiempo

$$O\left(\frac{XY}{\varepsilon^2}\log\frac{1}{\delta}\right).$$

Demostración. El algoritmo consiste en samplear independientemente m órdenes topológicos uniformes $\pi^{(1)}, \ldots, \pi^{(m)}$ y devolver el promedio

$$\widehat{ASV} = \frac{1}{m}\sum_{j=1}^{m}\left[\nu_{M,e,\mathrm{Pr}}(\pi^{(j)}_{<i} \cup \{x_i\}) - \nu_{M,e,\mathrm{Pr}}(\pi^{(j)}_{<i})\right].$$

Cada muestra requiere un sampleo de costo X y una cantidad constante de evaluaciones de la función característica, cada una de costo Y. El tamaño muestral necesario para garantizar error aditivo ε con confianza $1-\delta$ se obtiene mediante una cota estándar de concentración, lo que da

$$m = O\left(\frac{1}{\varepsilon^2}\log\frac{1}{\delta}\right).$$

Multiplicando por el costo por muestra se obtiene la cota deseada.

El resultado anterior abstrae completamente el problema en dos subrutinas: sampleo y evaluación de promedios. Una formulación análoga vale para clases de equivalencia.

Theorem 5. *Sea G un DAG y sea x_i un feature. Supongamos que existe un algoritmo que samplea una clase de equivalencia $[\pi]_R \in eqCl(G, x_i)$ con probabilidad proporcional a $|[\pi]_R|$ en tiempo X, y que la cantidad*

$$\nu_{M,e,\mathrm{Pr}}(L([\pi]_R))$$

puede calcularse en tiempo Y. Entonces, para cualquier $\varepsilon, \delta \in (0,1)$, existe un algoritmo que aproxima $Shap^{assym}_{M,e,\mathrm{Pr}}(x_i)$ con error aditivo a lo sumo ε y probabilidad al menos $1-\delta$ en tiempo

$$O\left(\frac{XY}{\varepsilon^2}\log\frac{1}{\delta}\right).$$

Demostración. La prueba es idéntica a la del Teorema 4, reemplazando órdenes topológicos por clases de equivalencia muestreadas con la distribución inducida por sus tamaños.

Estos resultados muestran que la complejidad del algoritmo aproximado factoriza de forma natural en dos componentes independientes: el costo de obtener una muestra y el costo de evaluar la predicción promedio asociada a ella.

3.6. Instanciaciones particulares

Los Teoremas 4 y 5 son útiles porque permiten derivar algoritmos concretos una vez que se dispone de procedimientos eficientes para samplear y para calcular predicciones promedio.

En nuestro caso, la familia de modelos de mayor interés es la de árboles de decisión. Para esta familia desarrollamos algoritmos polinomiales para calcular

$$\nu_{M,e,\mathrm{Pr}}(S)$$

cuando la distribución Pr está dada por una red bayesiana con estructura restringida. En particular, en la sección anterior mostramos que, si la distribución es un polytree y el modelo es un árbol de decisión, entonces la predicción promedio puede calcularse en tiempo polinomial. En consecuencia, en este escenario el costo Y de los Teoremas 4 y 5 es polinomial.

Esto da lugar a un esquema general para aproximar ASV: siempre que podamos samplear eficientemente órdenes topológicos o clases de equivalencia, obtenemos un algoritmo aproximado cuyo costo total es polinomial en el tamaño de la instancia y en los parámetros de precisión.

Por el contrario, cuando no se dispone de un algoritmo eficiente para calcular la predicción promedio, el enfoque anterior deja de ser útil. De hecho, el Teorema **??** sugiere que la tractabilidad de ASV está estrechamente ligada a la posibilidad de calcular eficientemente dichas esperanzas condicionadas. En particular, para familias de modelos en las que la predicción promedio es intratable, no cabe esperar un algoritmo aproximado eficiente basado en sampleo que evite completamente este cuello de botella, ya que cada muestra requiere precisamente evaluar una contribución marginal.

En resumen, los algoritmos de sampleo son especialmente relevantes en aquellos casos en los que:

1. el sampleo puede hacerse eficientemente;
2. el cálculo de promedios es polinomial;
3. el número de clases u órdenes exactos es demasiado grande como para justificar un enfoque exhaustivo.

Este es exactamente el régimen en el que trabajaremos en las subsecciones siguientes.

3.7. Sampleo uniforme de órdenes topológicos en polytrees

Nos concentramos ahora en el problema de samplear órdenes topológicos uniformemente. Nuestro primer acercamiento consistió en generar todos los órdenes topológicos mediante un algoritmo determinístico, como el de Knuth Knuth y Szwarcfiter, 1974, e ir devolviéndolos a medida que se producían. Sin embargo, este enfoque no resulta adecuado como procedimiento de sampleo: al ser determinístico, siempre devuelve los mismos órdenes en el mismo orden, y además no refleja la distribución uniforme deseada cuando distintos nodos fuente aparecen con frecuencias muy diferentes como comienzo de un orden topológico.

La idea correcta es samplear recursivamente el primer nodo del orden según una distribución proporcional a la cantidad de órdenes topológicos que comienzan con cada nodo fuente. Más precisamente, si S es el conjunto de nodos fuente de un DAG D, entonces para cada $s \in S$ definimos

$$p(s) = \frac{\#topos(D - \{s\})}{\#topos(D)},$$

donde $\#topos(D - \{s\})$ es la cantidad de órdenes topológicos de D que comienzan con s. El algoritmo elige el primer nodo del orden de acuerdo con esta distribución y luego continúa recursivamente sobre el grafo restante.

Algorithm 1 SampleoTopoSort(D)

---

1. Sea S el conjunto de nodos fuente de D.
2. Para cada $s \in S$, calcular la cantidad de órdenes topológicos de $D - \{s\}$ y definir

$$p(s) = \frac{\#topos(D - \{s\})}{\#topos(D)}.$$

3. Samplear un nodo $s^\star \in S$ según la distribución p.
4. Devolver $s^\star$ concatenado con `SampleoTopoSort`$(D - \{s^\star\})$.

La intuición del algoritmo se muestra en la Figura 5: si un nodo fuente aparece como primer elemento en una fracción mayor de los órdenes topológicos, entonces debe ser elegido con mayor probabilidad.

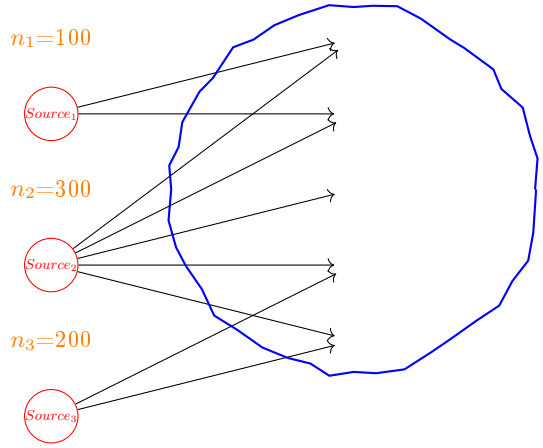


Figura 5: Comienzo del algoritmo de sampleo: cada nodo fuente se elige con probabilidad proporcional a la cantidad de órdenes topológicos que comienzan con él.

El punto crítico es que este algoritmo requiere contar la cantidad de órdenes topológicos de subgrafos. En el caso de *dtrees* ya disponíamos de una fórmula recursiva para ese conteo, pero nuestro objetivo es tratar con una familia más general. Por ello desarrollamos un algoritmo adicional para contar órdenes topológicos de polytrees arbitrarios, basado en un recorrido DFS sobre el grafo subyacente y en el mantenimiento de la información de posiciones posibles de cada nodo.

El resultado principal de esta parte es el siguiente.

Theorem 6. *Sea G un polytree de grado acotado. Entonces existe un algoritmo que samplea órdenes topológicos uniformemente de G en tiempo cuasi-polinomial, y por lo tanto, si la función característica puede evaluarse en tiempo polinomial, el valor de ASV puede aproximarse con error aditivo ε y confianza $1-\delta$ en tiempo cuasi-polinomial en el tamaño de la entrada, ε^{-1} y $\log(1/\delta)$.*

Demostración (Idea de la prueba). El algoritmo de sampleo uniforme reduce el problema a contar órdenes topológicos de subgrafos. Para polytrees de grado acotado, el algoritmo de conteo desarrollado en esta tesis tiene complejidad cuasi-polinomial. Aplicando el Teorema 4, se obtiene inmediatamente la cota para el algoritmo aproximado de ASV.

El Teorema 6 constituye nuestro resultado más general basado en sampleo de órdenes topológicos. Su importancia radica en que extiende el esquema de aproximación más allá de los casos exactos previamente analizados: aun cuando no podamos computar todas las clases de equivalencia ni todos los órdenes topológicos, seguimos pudiendo estimar ASV con buena precisión siempre que el conteo y el sampleo permanezcan tratables.

3.8. ASV exacto

Recordemos la fórmula introducida en la sección **??** para ASV, utilizando nuestra heurística.

$$\phi_i^{assym}(\nu) = \sum_{\pi \in perm(\mathcal{I})} w(\pi)\left[\nu(\pi_{<i} \cup i) - \nu(\pi_{<i})\right] =$$

$$\frac{1}{|topos(G)|} \sum_{[\pi]_R \in eqCl(G,x_i)} \left(\nu(\pi_{<i} \cup \{x_i\}) - \nu(\pi_{<i})\right) \cdot |[\pi]_R|$$

Dado un DAG G, un nodo $x_i \in V(G)$ y una función característica ν. Nuestro algoritmo completo queda así entonces:

- Calculamos $eqClass$ a través de $eqClassSizes(G, x_i)$, el algoritmo que introducimos en la sección **??**
- Luego para cada clase de equivalencia vamos a calcular el promedio teniendo en cuenta los features previos a x_i. Para realizar el promedio vamos a utilizar el algoritmo de la sección **??**. (En el caso de que el modelo no sea un árbol, podemos aproximarlo utilizando Monte Carlo, pero el algoritmo dejaría de ser exacto)

- Por último sumamos los resultados para obtener él $Shap^{assym}$ para el feature i.

Las modificaciones que introducimos son para mejorar la performance del mismo, sin modificar los resultados obtenidos a diferencia del enfoque aproximado. En la sección a continuación vamos a ver cuál es la mejora respecto al enfoque naive y el tiempo que tarda para distintas redes.

3.9. ASV aproximado

Este algoritmo es igual al anterior. La única diferencia que tiene es respecto a cómo se calculan las clases de equivalencia, $eqClass(G, x_i)$, puesto que ahora las vamos a aproximar. Además, vamos a tener que agregar un parámetro extra, para definir la cantidad de órdenes topológicos que queremos generar. Primero utilizamos el algoritmo **??** para samplear los órdenes topológicos. Esto nos va a permitir calcular él ASV para polytrees y no simplemente *dtrees*. Luego procesamos estos órdenes al igual que en la sección **??**, para obtener las clases de equivalencia. Una vez obtenidas las clases de equivalencia, repetimos el mismo proceso del algoritmo exacto.

1. Resultado de que ASV se puede calcular en algunos casos donde el otro no se podía (el caso se la naiva bayes digo). Creo que no hay apendice, asi que diria de, en principio, poner todas las demos en el main text. Tienen que estar prolijas.
2. Idea de las clases de equivalencia. Definición y relación con la cantidad de órdenes topológicos.
3. Descripción general del algoritmo basado en clases de equivalencias para cómputo exacto.
4. Instanciar este algoritmo en el caso de árboles y dar la complejidad.
5. Introducir la opción de samplear para aproximar el ASV, tanto clases de equivalencia como ordenes topológicos.
6. Dar dos resultados generales diciendo "Si podes samplear clases / ordenes en X tiempo y calcular promedios en Y, la complejidad es f(XY)".
7. Instanciar este resultado en casos particulares. Hablar de los algoritmos para promedios de arboles. Capaz el resultado de imposibilidad.
8. Dar el resultado basado en sampleo de ordenes topologicos en general.

4. Experimental results

1. Comparación entre clases de equivalencia y ordenes topologicos.
2. Algun resultado de performance sobre algoritmos de clases de equivalencias.
3. Si podemos, estaria bueno implementar el otro algoritmo basado en sampleo de ordenes topologicos.
4. No me parece necesario contar la performance de los algoritmos que calculan promedios, aunque si hay lugar podemos.
5. Si hay lugar tambien podriamos hacer una comparacion de shapley y ASV (no me queda claro si hay o no pq son 14 paginas pero el formato es una verga)

5. Conclusions

6. First Section

6.1. A Subsection Sample

Please note that the first paragraph of a section or subsection is not indented. The first paragraph that follows a table, figure, equation etc. does not need an indent, either.

Subsequent paragraphs, however, are indented.

Sample Heading (Third Level) Only two levels of headings should be numbered. Lower level headings remain unnumbered; they are formatted as run-in headings.

Sample Heading (Fourth Level) The contribution should contain no more than four levels of headings. Table 1 gives a summary of all heading levels.

Tabla 1: Table captions should be placed above the tables.

Heading level	Example	Font size and style
Title (centered)	**Lecture Notes**	14 point, bold
1st-level heading	**1 Introduction**	12 point, bold
2nd-level heading	**2.1 Printing Area**	10 point, bold
3rd-level heading	**Run-in Heading in Bold.** Text follows	10 point, bold
4th-level heading	*Lowest Level Heading.* Text follows	10 point, italic

Demostración. Proofs, examples, and remarks have the initial word in italics, while the following text appears in normal font.

For citations of references, we use the APA 7 convention. The following bibliography provides a sample reference list with entries for journal articles (**ref_article1**), an LNCS chapter (**ref_lncs1**), a book (**ref_book1**), proceedings without editors (**ref_proc1**), and a homepage (**ref_url1**). Multiple citations are grouped (**ref_article1**; **ref_lncs1**; **ref_book1**), (**ref_article1**; **ref_book1**; **ref_proc1**; **ref_url1**).

Acknowledgments. A bold run-in heading in small font size at the end of the paper is used for general acknowledgments, for example: This study was funded by X (grant number Y).

Disclosure of Interests. It is now necessary to declare any competing interests or to specifically state that the authors have no competing interests. Please place the statement with a bold run-in heading in small font size beneath the (optional) acknowledgments, for example: The authors have no competing interests to declare that are relevant to the content of this article. Or: Author A has received research grants from Company W. Author B has received a speaker honorarium from Company X and owns stock in Company Y. Author C is a member of committee Z.

Referencias

Arenas, M., Barceló, P., Bertossi, L., & Monet, M. (2021). The tractability of Shap-score-based explanations for classification over deterministic and decomposable Boolean circuits. *Proceedings of the AAAI Conference on Artificial intelligence*, *35*, 6670-6678.

Arenas, M., Barceló, P., Bertossi, L., & Monet, M. (2023). On the complexity of Shap-score-based explanations: Tractability via knowledge compilation and non-approximability results. *Journal of Machine Learning Research*, *24*(63), 1-58.

Epoch AI. (2024, junio). Data on Large-Scale Ai Models [Accessed: 2025-04-14]. https://epoch.ai/data/large-scale-ai-models

et. al, E. G. (2024). FrontierMath: A Benchmark for Evaluating Advanced Mathematical Reasoning in Ai. https://arxiv.org/abs/2411.04872

Frye, C., Rowat, C., & Feige, I. (2019). Asymmetric Shapley values: incorporating causal knowledge into model-agnostic explainability. *arXiv preprint arXiv:1910.06358*.

Fryer, D., Strümke, I., & Nguyen, H. (2021). Shapley values for feature selection: The good, the bad, and the axioms. *Ieee Access*, *9*, 144352-144360.

Huang, X., & Marques-Silva, J. (2023). The inadequacy of Shapley values for explainability. *arXiv preprint arXiv:2302.08160*.

Knuth, D. E., & Szwarcfiter, J. L. (1974). A structured program to generate all topological sorting arrangements. *Information Processing Letters*, *2*(6), 153-157. https://doi.org/https://doi.org/10.1016/0020-0190(74)90001-5

Lipton, Z. C. (2017). The Mythos of Model Interpretability. https://arxiv.org/abs/1606.03490

Lundberg, S., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. https://arxiv.org/abs/1705.07874

Marques-Silva, J. (2023). Logic-based explainability in machine learning. En *Reasoning Web. Causality, Explanations and Declarative Knowledge: 18th International Summer School 2022, Berlin, Germany, September 27–30, 2022, Tutorial Lectures* (pp. 24-104). Springer.

Marzouk, R., Bassan, S., Katz, G., & de la Higuera, C. (2025). On the Computational Tractability of the (Many) Shapley Values. https://arxiv.org/abs/2502.12295

Miller, T. (2019). Explanation in Artificial intelligence: Insights from the social sciences. *Artificial Intelligence*, *267*, 1-38. https://doi.org/https://doi.org/10.1016/j.artint.2018.07.007

Pearl, J. (1986). Fusion, Propagation, and Structuring in Belief Networks. *Artificial Intelligence*, *29*(3), 241-288.

Shapley, L. S., et al. (1953). A value for n-person games. *Classics in game theory*.

Van den Broeck, G., Lykov, A., Schleich, M., & Suciu, D. (2022). On the tractability of Shap explanations. *Journal of Artificial Intelligence Research*, *74*, 851-886.